

Pytorch

- ① Dataset & Dataloader
- ② Embedding



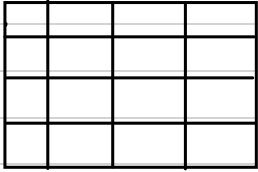
Tensors in Pytorch

Rank 0 
scalar

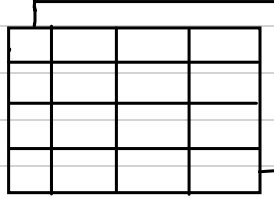
Rank 1 (vector)



Rank 2 (matrix)



Rank 3



```
import torch
print(torch.__version__)
```

```
if torch.cuda.is_available():
    print("GPU is available")
    print(f"using GPU: {torch.cuda.get_device_name(0)}")
else:
    print("GPU not available")
```

① Creating & using Tensor

```
a = torch.empty(2,3)
b = torch.zeros(2,3)
c = torch.ones(2,3)
d = torch.rand(2,3)
type(a) # Tensor
a.shape #
```

```
torch.manual_seed(100)
torch.rand(2,3)
```

Using Tensor

```
x = torch.tensor([1,2],[3,4])
y = torch.arange(0,10,2)
z = torch.linspace(0,10,10)
w = torch.eye(5) # identity matrix 5x5
p = torch.full((3,3),5)
```

```
torch.empty_like(x)
torch.zeros_like(x)
torch.ones_like(x)
torch.rand_like(x)
```

② DataType

- `x.dtype`
- `torch.tensor([1.0, 2.0], dtype=torch.int32)`

③ Mathematical Operations

- Addition : `x + 2`
- Subtraction : `x - 2`
- Multiplication : `x * 3`
- Division : `x / 3`
- Int Division : `(x * 100) // 3`
- power : `x ** 2`
- Absolute (`abs`)
- round (`round`)
- ceil
- floor
- clamp

Clamp

`x = torch.tensor([1.9, 2.3, 3.7, 4.4])`

`torch.clamp(x, min=2, max=3)`

$\Rightarrow$ # `[2.0, 2.3, 3.0, 3.0]`

Reduction

sum median std max
mean product var min

`x = torch.randint(size=(2,3), low=0, high=10)` # $\begin{bmatrix} 5 & 5 & 8 \\ 8 & 5 & 0 \end{bmatrix}$

`torch.sum(x)`

`torch.sum(x, dim=0)`

`torch.sum(x, dim=1)`

`torch.mean(x)`

`torch.mean(x, dim=0)`

• `std(x)` # std. deviation

• `var(x)` # variance

• `argmax` ← position of index of biggest number

• `argmin` ← position of index of lowest number

`torch.argmax(x)`

`torch.argmin(x)`

Matrix Multiplication

• `torch.matmul(m1, m2)` # Matrix multiplication

• `torch.dot(v1, v2)` # Dot product

• `torch.transpose(m1, 0, 1)` # ← ??

$$\begin{pmatrix} 3 & 7 & 8 \\ 7 & 9 & 9 \end{pmatrix} \Rightarrow \begin{pmatrix} 3 & 7 \\ 7 & 9 \\ 8 & 9 \end{pmatrix}$$

• `torch.det(m1)` # Determinant

• `torch.inverse(m1)`

GPU Performance Comparison

```
import time  
SIZE = 10000
```

```
mat_cpu1 = torch.randn(SIZE, SIZE)  
mat_cpu2 = torch.randn(SIZE, SIZE)
```

```
start_time = time.time()  
result = torch.matmul(mat_cpu1, mat_cpu2)  
cpu_time = time.time() - start_time  
print(f"Time on CPU: {cpu_time:.4f} sec")
```

move matrix to GPU

```
mat1 = mat_cpu1.to('cuda')  
mat2 = mat_cpu2.to('cuda')
```

Measure time on GPU

```
start = time.now()  
result = torch.matmul(mat1, mat2)  
torch.cuda.synchronize()  
time = time.time() - start_time
```

Autograd

⑥

$$\left. \begin{aligned} y &= x^2 \\ \frac{dy}{dx} &= 2x \end{aligned} \right\}$$

```
def dy_dx(x):
    return 2*x
```

$$\left. \begin{aligned} y &= x^2 \\ z &= \sin(y) \end{aligned} \right\} \left\{ \begin{aligned} \frac{dz}{dx} &= \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x} \\ &= \cos y \cdot 2x \\ &= 2x \cos(y) \\ &= 2x \cos(x^2) \end{aligned} \right\}$$

```
def dz_dx(x):
    return 2*x * math.cos(x**2)
```

$$\left. \begin{aligned} y &= x^2 \\ z &= \sin(y) \\ u &= e^z \end{aligned} \right\} \frac{du}{dx} = \frac{\partial u}{\partial z} \cdot \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Ex:

cgpa	placed
9.11	1
8.9	1
7	0
6.56	1
4.56	0



```
x = torch.tensor(3.0, requires_grad=True)
```

```
y = x ** 2
```

```
y # Tensor(9, grad_fn=<PowBackward0>)
```



```
y.backward() #
```

```
x.grad # 6
```

Ex 2 :

$$\left. \begin{array}{l} y = x^2 \\ z = \sin(y) \end{array} \right\} \frac{dy}{dx} = ?$$

```
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
z = torch.sin(y)
```

x : tensor(3, requires_grad=True)

y : tensor(9, grad_fn=<PowBackward0>)

z : tensor(0.4121, grad_fn=<SinBackward0>)



z.backward()

x.grad() # - 7

y.grad() ~~X~~ $\leftarrow$ Not a leaf node

Ex 3

$$\left. \begin{array}{l} x = [x_1, x_2, x_3] \\ y = x^2 \cdot \text{mean}() \end{array} \right\} \boxed{y = \frac{x_1^2 + x_2^2 + x_3^2}{3}} \leftarrow \text{Multi variable Function}$$

$$y = f(x_1, x_2, x_3)$$

$$\begin{aligned} \frac{\partial y}{\partial x_1} &= \frac{2x_1}{3} ; \frac{\partial y}{\partial x_2} = \frac{2x_2}{3} ; \frac{\partial y}{\partial x_3} = \frac{2x_3}{3} \\ &= \frac{2 \cdot 1}{3} , \quad = \frac{2 \cdot 2}{3} \quad = \frac{2 \cdot 3}{3} \\ &= 2/3 \quad = 4/3 \quad = 2 \end{aligned}$$

```
x = torch.tensor([1, 2, 3], requires_grad=True)
y = (x ** 2).mean()
y.backward()
x.grad() # [0.667, 1.33, 2.00]
```

Clearing Gradient

x.grad.zero_() $\leftarrow$ underscore represent in-place

Pytorch Training Pipeline